

AN2DL - First Challenge Report

overfittinghappens

Simone Sartori, Lorenzo Roberto Serraglio, Francesca Viggiani

simonesartori, loreserraglio, francescaviggiani

303656, 214128, 299452

June 21, 2026

1 Introduction

This project focuses on *dataset classification* using deep learning techniques. Our approach consisted of:

- Analyzing, cleaning and preprocessing the dataset;
- Developing a basic custom **Neural Network** from scratch;
- Gradually improving the NN with small adjustments and more advanced techniques.

2 Problem Analysis

The provided dataset consists of repeated time-series observations collected on 661 different subjects, who could be either ordinary people or pirates. For each subject were recorded 160 observations over time, regarding their physical condition, with the goal of classifying their true level of pain into 3 categories: no pain, low pain and high pain. The features include: 4 **pain survey responses**, some **categorical attributes** such as number of legs, hands and eyes and 31 continuous **measurements of body joint angles**. In order to simplify the dataset, we removed the columns with very low standard deviation, since features that barely

vary across subjects usually do not contribute to the model's predictive ability. We also converted the categorical variables into numerical ones in order to be used by the neural network. For example, the variable "**n_legs**", which originally contained the values "*two*" or "*one + peg leg*", was transformed into a binary variable "**two_legs**". We also noticed a significant **class imbalance** (Figure 1) that we tried to address later in the project.

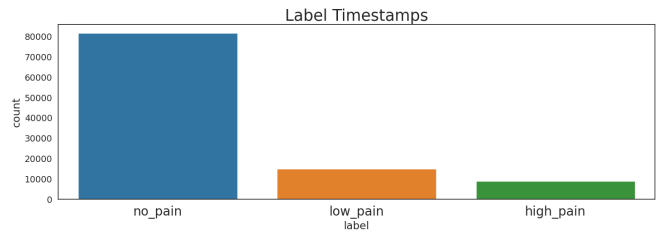


Figure 1: Distribution of the timestamps

3 Method

3.1 Basic RNN

After exploring the data, we split it into a training set (80%) and a validation set (20%), and we performed some pre-processing such as **min-max normalization** on the "joint" columns. Since the dataset contains time-series data, we applied windowing techniques to better capture temporal structure. At this point, we started by building a simple

NN with 2 hidden layers using a **RNN** architecture. In order to reduce overfitting and improve the generalization over unseen data, we used early stopping and **L2 regularization** techniques, and we set a dropout rate. With this basic model we obtained 81.57% validation accuracy and 85.98% F1 score on the test.

3.2 CrossValidation & Hyperparameters Tuning

To improve the performance of our first model, we wanted to be able to find the best parameters and hyperparameters configuration. We added a **K shuffle split cross validation** function which performs K different splits of the dataset and uses a different one of them as a validation set on each iteration. The final performance is then computed as the average across all folds. In our model we used this function along with a **grid search function**, which combines vectors of parameters and computes all the possible configurations. This let us train and evaluate our model with different sets of parameters, so that we could find the ones that gave the best performance. We tested different neural network architectures and found that **GRU** performed the best for our task. Basic RNNs were unable to learn the necessary long-term dependencies, likely due to vanishing gradients, and *LSMT* turned out to be less effective than GRU in managing our small dataset with long sequences. Furthermore, a **bidirectional** has been essential to enhance the performance. Due to its capability of capturing the full context of a movement, it distinguishes better a normal gesture from a painful one. Then we tried different combinations of hyperparameters, such as *learning rate*, *window size* and *stride*, number of *hidden layers*. In the end we tried using *L1 regularization* as a weight regularization technique, but got worse performance compared with **L2 regularization**. Once we found our best configuration, we re-trained our model, using as starting weights the ones of the best configuration (warm start). As a consequence of that, we used in retraining a learning rate 10 ten times smaller. We evaluated our model on the original Train and Validation sets and then used it to get a prediction on the test set. With this model we got 90.40% validation accuracy and 92.57% F1 score on the test set.

3.3 Fine tuning

Once the hyperparameters of our model were defined, we focused on further techniques with the aim of managing the characteristics of the dataset in the best way, and of preventing overfitting.

- **Weighted loss:** To handle the significant class imbalance in the dataset we proceeded applying weighted loss. This technique consists in modifying the loss function to give different importance to classes during training, multiplying the loss from each class by a weight. This helps the model learn to recognize underrepresented classes.
- **Focal loss:** To achieve even better results, we combined weighted loss with focal loss. Unlike standard Cross-Entropy, which treats all samples equally, Focal Loss introduces a modulating factor that decreases the contribution of well-classified observations.
- **Gradient Clipping:** During the training of our models, we noticed a potentially dangerous instability in the plot of the validation loss and F1 score, which can lead to unreliable predictions. To handle this, we limited the magnitude of the gradients during the training, avoiding exploding gradients. When the norm of the gradient exceeds the set threshold (in our case 1), the gradient is normalized.
- **Attention:** We used the attention mechanism to make our neural network focus dynamically on the most important parts of the input data, giving more weight to the critical moments. It works by assigning weights to different input elements based on their importance for the current prediction.
- **Windowing and correlation:** To better understand the temporal structure of the dataset, we computed the autocorrelation of each numeric feature across time. Autocorrelation allows us to measure how strongly a signal is correlated with itself at different time lags. For each subject and for each feature, we extracted the time-series, normalized it by removing the mean (to improve numerical stability), and computed the autocorrelation curve using only the positive lags up to a

Table 1: Performance comparison

Model	Validation Accuracy (%)	Validation F1 (%)	Test F1(%)
Basic RNN	81.57	77.61	85.98
K-cv & hyperparameters tuning	90.40	90.20	92.57
Final Model	92.85	92.86	95.14

maximum of 150 timestamps. Constant or extremely short sequences were skipped to avoid degeneracies. The analysis of the curve (Figure 2) suggested that we could use a smaller window size compared to the one we got from the hyperparameters tuning, that was set to 80, because the curve is stable even for small timestamps. Given the plot we chose a window size of 30.

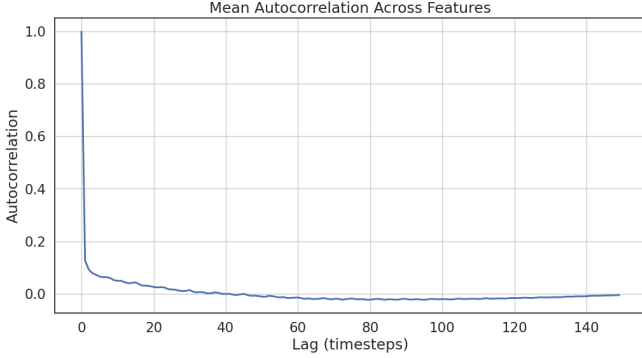


Figure 2: Autocorrelation curve

4 Failed experiments

During our experiments we tried different techniques. However not all of them resulted into a better performance of the model:

- **Oversampling:** It is another technique to handle class imbalance by increasing the number of samples in the minority classes. Probably due to overfitting, or to the creation of

samples that don't reflect the data distribution, this strategy worsened the generalization of the model on unseen data.

- **Label smoothing:** It consist in redistributing probability mass to non-target classes to reduce the model confidence. We did not employ label smoothing as it proved incompatible with focal loss.

5 Results

Through systematic experimentation with various hyperparameters and regularization strategies, we were able to improve our first model and to achieve a 90.40% validation accuracy and a 95.14% F1 score on the test data (Table 1)

6 Conclusions

Our work shows how it is possible to tackle the task of time sequence classification, combining a proper analysis of the dataset, to understand its key properties, and the usage of the right techniques, to handle the challenges of time sequence classification. There's still room to improve our model. For example, identifying larger window sizes could have lead to the implementation of 1D convolutional layers with the goal of improving the model's ability to capture local temporal patterns in the joint angle sequences.